

1. What is Software Quality?

Software quality is the degree to which a software system:

- Produces correct results.
- Performs efficiently.
- Is secure and reliable.
- Is easy to use.
- Is available when needed.

Key Formula: Quality = Meeting Requirements + Satisfying Users

Key Terminology: Error, Defect, Failure

Understanding the distinction between these terms is crucial. They represent the journey of a mistake from inception to impact.

Term	Definition	When it Occurs
Error	A mistake made by a developer or a human.	During development (requirements, design, coding).
Defect (Bug)	A incorrect code or design	Present in the software artifact (code, document).
Failure	The software does not work as expected during execution.	At runtime, when the defective code is executed.

Flow: Human Mistake (Error) → Bug in Software (Defect) → Wrong System Behavior (Failure)

- The cause of a failure inside the system is called Fault

2. What is Verification & Validation (V&V)?

- **Verification:** "Are we building the product right?" It ensures the software correctly implements a specific function. (e.g., reviewing code, static analysis).
- **Validation:** "Are we building the right product?" It ensures the software meets the user's actual needs and requirements. (e.g., system testing, user acceptance testing).

3. When is Software Ready for Release?

Contrary to common beliefs (e.g., "when we can't find bugs," "when testing is finished"), release is determined by more strategic factors:

- Requirements are satisfied.
- Critical defects are fixed.
- Quality meets agreed standards.
- Risk level is acceptable.
- Stakeholders approve release.

4. The Cost of Quality & Fixing Defects

Cost of Quality (CoQ)

This is the money spent beyond building the product correctly the first time. It is divided into three categories:

1. **Prevention Cost:** Costs incurred to prevent defects.

- Developer training, good design practices, coding standards, proper planning.

2. **Appraisal Cost:** Costs to evaluate and inspect the product.

- Reviews, inspections, testing activities.

3. **Failure Cost:** Costs resulting from defects.

- Fixing defects, handling customer complaints, rework, and maintenance.

The Exponential Cost of Fixing Defects

The cost to fix a defect grows exponentially the later it is found in the software development life cycle (SDLC).

- **Specification Phase:** 1 unit (cheapest)
- **Design Phase:** 10 units
- **Coding Phase:** 100 units
- **Testing Phase:** 1000 units
- **After Release:** Unpredictable & very high (most expensive)

Conclusion: Detecting defects early is a primary objective of V&V.

5. The Limitations of Testing

Can we test everything? No. Complete testing is theoretically impossible due to:

- **Huge number of possible inputs** for any non-trivial system.
- **Many execution paths** through the code.

- **Limited time and budget.**

This highlights the need for careful planning and risk-based testing (ILO 3 & 4).

6. Software Quality Attributes

Quality attributes are non-functional requirements that are critical to a system's success. They can be split into two categories for exams:

User-Facing Attributes

- **Performance:** Ability to respond within required time limits.
- **Security:** Protects information from unauthorized access.
- **Reliability:** Operates consistently without unexpected failure.
- **Availability:** System is accessible and operational when needed.
- **Usability:** Easy to learn, use, and understand.
- **Scalability:** Handles increasing users or workload efficiently.

Developer/System-Facing Attributes

- **Modifiability (Maintainability):** Ease of fixing defects and adding new features.
- **Testability:** Ease of detecting faults through testing.
- **Interoperability:** Ability to communicate and work with other systems.

- **Portability:** Ability to run on different platforms or environments.

7. Common Causes of Software Failures

- Poor requirements
- Design mistakes
- Coding errors
- Insufficient testing
- Communication gaps

8. Core Definitions & Objectives

- **V&V Definition:** Verification and Validation are systematic activities performed to evaluate software quality throughout the development lifecycle. They are core components of Software Quality Assurance (SQA).
- **Primary Goals of V&V:**
 1. Ensure the software **conforms to its specifications**.
 2. Ensure the software **fulfills user needs and intended use**.
 3. Ensure the software is **dependable prior to release**.

9. The Fundamental Distinction: Verification vs. Validation

This is the most critical concept of the lecture. The definitions are attributed to Barry Boehm.

Aspect	Verification	Validation
Key Question	"Are we building the product right?"	"Are we building the right product?"
Focus	Conformance to specification.	Satisfaction of user needs and real-world requirements.
Perspective	Technical: Checks if the implementation matches its input specification.	Practical: Checks if the system solves the real problem and fulfills stakeholder expectations.
Activities	Requirements Reviews, Design Reviews, Code Walkthroughs, Code Inspections, Static Analysis.	Unit Testing, Integration Testing, System Testing, User Acceptance Testing (UAT), Alpha/Beta Testing.
Characteristics	Primarily static (no code execution). Focuses on defect prevention .	Primarily dynamic (code execution). Focuses on observing behavior and detecting runtime failures .
Key Outcome	Shows that software is dependable (works as intended).	Shows that software is useful (meets user needs).

Verification

Verification is the process of evaluating whether the work-products of a development phase satisfy the **conditions and requirements defined at the start of that phase**.

What Verification Examines

- Is the implementation consistent with its specification?
- Does the system behave as defined under specified conditions?
- Are data structures, interfaces, and logic correctly implemented?

Specification and Implementation as Roles

- “Specification” and “Implementation” are relative terms.
- Examples:
 - Requirements $\leftrightarrow$ Design
 - Design $\leftrightarrow$ Source Code
 - Test Cases $\leftrightarrow$ Requirements
 - Source Code $\leftrightarrow$ User Documentation
- At each stage, the output must **conform to its input specification**.

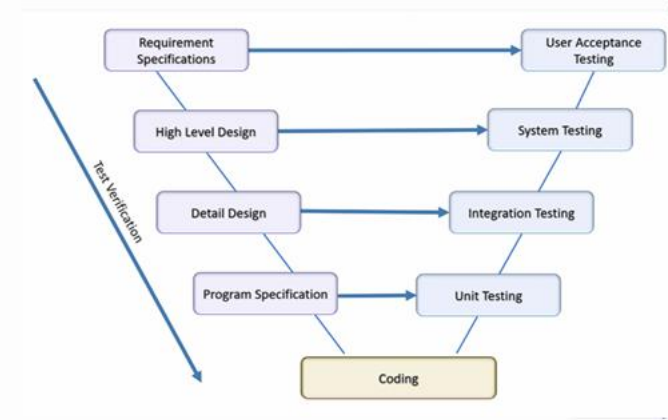
Typical Verification Activities

- Requirements Reviews
- Design Reviews

- Code Walkthroughs
- Code Inspections
- Static Analysis

Characteristics

- Primarily static and analytical
- Does not require program execution
- Focuses on defect prevention
- Checked after each development activity against the previous activity.



Validation

Validation is the process of evaluating a system during or at the end of development to determine whether it satisfies user needs and intended operational requirements.

Focus of Validation

- Does the system solve the real problem?
- Does it fulfil stakeholder expectations?
- Does it operate effectively in its intended environment?

Scope and Activities of Validation

Real-World Perspective

Validation examines:

- System behaviour under realistic conditions
- User interaction and satisfaction
- Practical usability and performance

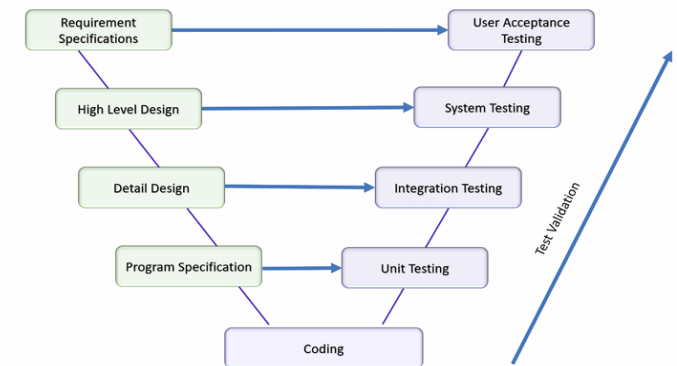
Typical Validation Activities

- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing (UAT)

- Alpha and Beta Testing

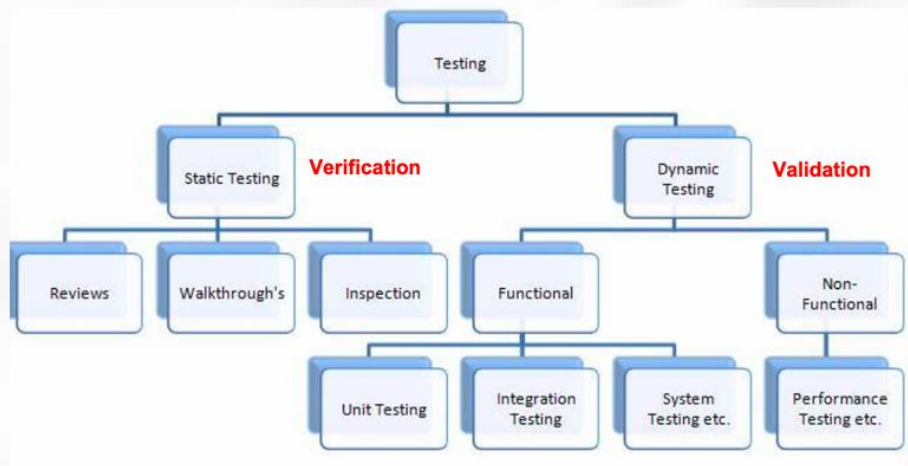
Characteristics

- Primarily dynamic and execution-based
- Focuses on observing system behaviour
- Detects runtime failures and requirement gaps
- Checked against user needs and requirements.



3. V&V Across the Software Lifecycle

A key principle is that V&V is **not a final phase** but an activity that starts early and continues after release.



a. Early Start (Shift Left)

V&V can begin as soon as the project starts:

- **Feasibility studies:** Must consider quality assessment.
- **Requirements:** Can be used to derive test cases.
- **Design:** Can be verified against requirements.
- **Code:** Can be verified against design and requirements.
- **Stakeholders:** Feedback can be sought at any time.

b. When to Stop V&V? (The "Good Enough" Principle)

- **Reality:** Finding all faults is nearly impossible.
- **Approach:** Instead of aiming for perfection, establish criteria for acceptance. Decide "how good is good enough?"
- **Methods:**

- **Quantitative:** Measure dependability (e.g., reliability metrics) and set a threshold (metric > threshold).
- **Qualitative:** Use Alpha/Beta testing to gather user feedback and judge product readiness.

c. V&V After Release (Maintenance Phase)

V&V does not end with release. Software evolves, requiring:

- **Regression Testing:** When code changes, rerun tests to ensure existing functionality still works.
- **Test Case Maintenance:** Retain tests that exposed faults to ensure they do not return.

4. Balancing V&V Effort

The level of V&V rigor depends on several factors:

- **Software Purpose:** More critical systems (e.g., medical devices, aircraft) require more rigorous V&V.
- **User Expectations:** Users may tolerate some bugs if the benefits outweigh the cost of failure.
- **Marketing Environment:** Pressure to be first to market may influence the depth of V&V.

5. Organizational Learning

V&V is also a process for continuous improvement:

- Collect data during development (fault information, bug reports, metrics).
- Classify faults into categories to identify common mistakes.
- Share information within the organization to avoid repeating the same errors in future projects.

What is V&V? Why do we need it?

Modern software systems are:

- **Large and complex**
- **Used in critical applications** (like hospitals, planes)
- **Always changing** (updates, new features)

So we must make sure software is:

- **Reliable**
- **Safe**
- **Secure**
- **Fit for its purpose**

Main Goal of V&V

To build confidence that the software is fit for its intended use.

Software **does not need to be 100% error-free.**

It just needs to be **reliable enough** for the environment it works in.

What Affects Confidence in Software?

Confidence depends on:

1. **How the software will be used**
2. **What happens if it fails**
3. **What users expect**

Different systems need different levels of confidence.

Example: Confidence Levels

System	Required Confidence
Online shopping website	Moderate
Banking transaction system	High
Air traffic control system	Extremely high

Three Factors That Control V&V Effort

1. **Software Purpose**
2. **User Expectations**
3. **System Criticality**

These decide how much testing and checking you need to do.

Factor 1: Software Purpose

The purpose tells you how reliable the software must be.

Examples:

- Mobile game → low risk
- Online banking system → high reliability needed
- Medical device software → extremely high reliability

Factor 2: User Expectations

Different users expect different things.

Example:

- Social media users may ignore small errors
- Banking customers expect **accurate and reliable** systems

Factor 3: System Criticality

It how serious the damage is if the system fails.

Three types:

Safety-critical systems

Failure can cause:

- Injury
- Death
- Environmental damage

Examples: Aircraft control, medical equipment, nuclear power plants

Require **extremely rigorous** V&V

Mission-critical systems

Failure can cause:

- Major operation disruption
- Loss of services

Examples: Banking systems, military communication, airline reservations

Business-critical systems

Failure mainly causes **financial loss or service disruption**

Examples: E-commerce, online booking, enterprise management

Why Find Defects Early?

The cost of fixing a defect **increases** as the project goes on.

Development Phase	Cost of Fixing Defect
Requirements	Low
Design	Medium
Implementation	High
Maintenance	Very High

Therefore, Early verification is important

Benefits of finding defects early:

- Reduces development cost
- Reduces maintenance work
- Improves product quality

Early V&V activities:

- Requirement reviews
- Design inspections
- Code reviews

The Limits of Testing

Testing cannot guarantee software is completely error-free.

Why?

- **Infinite input combinations** – you can't test everything
- **Complex interactions** between modules
- **Limited testing time**

Testing can only show that defects exist – not that they don't exist.

Constraints on V&V Activities

V&V is limited by:

- Time constraints
- Budget limitations
- System complexity
- Availability of skilled testers

Time Constraints

Software projects have strict deadlines.

So:

- Testing time may be limited
- Not all test cases can be run
- Must focus on **critical features first**

Budget Limitations

Testing needs resources like:

- Testing tools
- Infrastructure (computers, networks)
- Skilled testers

Organizations must balance:

Testing cost vs. cost of system failure

System Complexity

Modern systems may include:

- Cloud computing
- Distributed applications
- Microservices
- External APIs

Human Errors Happen

Software development involves human activities.

Errors can occur during:

- Requirements specification
- System design
- Implementation (coding)
- Testing

V&V helps find and reduce these human errors.

V&V and Software Evolution

Software changes over time because of:

- Changing user requirements
- Technology updates
- Security improvements
- Bug fixes

V&V makes sure the system stays reliable as it evolves.

During Maintenance: Watch for New Defects

Changes can introduce new defects.

So:

- V&V activities must be repeated after changes
- This ensures the system still works correctly

Regression Testing

Regression testing ensures that new changes do not break old functionality.

Example:

Updating the payment module should not break the login system or order processing.

Risk Based Testing

Because of constraints, testing must focus on **high-risk areas**.

Risk factors include:

- Critical system functions
- Security-sensitive modules
- Frequently changed components

Examples of high-risk components:

- Transaction processing
- User authentication
- Payment processing
- Database management

These areas need **extensive V&V**.

Effective V&V Requires:

- Early testing
- Continuous verification (keep checking)
- Risk-based testing strategies
- Skilled testing teams

Benefits of Good V&V:

- Improved software reliability
- Reduced development cost
- Increased customer confidence
- Reduced system failures

Remember: V&V Does NOT Remove All Defects

Even with strong V&V:

- Some defects may still remain
- Testing cannot cover every scenario
- Human errors may still happen

V&V aims to **reduce risk** – not eliminate it completely.

What is V&V Planning?

A **structured plan** that defines how V&V activities will be carried out.

- Developed at the **early stage** of the project lifecycle
- Includes:
 - V&V goals and objectives
 - Techniques and tools

- Roles and responsibilities
- Schedule and resources
- Acts as a **roadmap** to ensure software quality throughout development

Why is V&V Planning Important?

- Ensures **systematic execution** of V&V activities
- Helps identify **defects early** in development
- **Reduces cost** of fixing errors in later stages
- Improves **reliability and maintainability** of software
- Supports **project management and risk control**

Real-world example: NASA Mars Orbiter failed due to poor verification planning.

Fix the Requirement Activity

Given requirement: "User can login quickly"

Problems:

- "Quickly" is **vague**
- **Not measurable**
- **Cannot be tested properly**

Improved requirement:

"User should be able to login within 2 seconds after entering valid credentials"

Key qualities of a good requirement: Clear, Measurable, Testable

V&V Planning Stages Overview

Static vs Dynamic V&V (Planning Perspective)

Aspect	Static V&V	Dynamic V&V
Planning Stage	Early lifecycle stages	Later lifecycle stages
Focus	Document and code evaluation	Execution-based testing
Includes	Inspection, static analysis	Unit, integration, system testing

Static vs Dynamic Verification

Aspect	Static Verification	Dynamic Verification
Execution	Without executing the program	By executing the software
Purpose	Detects defects early	Detects runtime errors and behavior issues
Applied To	Documents, code	Software behavior at different testing levels

Static Verification Techniques

1. **Software inspection** – formal review
2. **Walkthroughs** – informal discussion
3. **Technical reviews** – semi-formal evaluation
4. **Static analysis** – tool-based checking

Software Inspection

- A **formal and structured** review technique.
- Conducted by a team with defined roles:
 - **Moderator**
 - **Reviewer**
 - **Recorder**
- Uses **checklists** to systematically identify defects
- Applied to **requirements, design, and code**
- Helps detect **logical and conceptual errors** early

Code Example: Login Function Issues

```
function login(username, password)
```

```
    if username == "admin"
```

```
        print("Login success")
```

```
    else
```

```
        print("Error")
```

Issues identified:

1. Password is **NOT checked**
2. **Hardcoded** username ("admin")
3. No **input validation**
4. No **security handling**

Fixed code:

```
function login(username, password)
```

```
    if username == "admin" AND password == "1234"
```

```
        print("Login success")
```

```
    else
```

```
        print("Invalid username or password")
```

What is Hardcoding?

Writing **fixed values directly in code**:

- if username == "admin" and password == "1234"

Problems:

- Not flexible
- Not secure
- Cannot handle multiple users

Real-world approach: Use a **database** to store usernames and passwords.

```
function login(username, password)
    user = database.getUser(username)
    if user != null AND user.password == password
        print("Login success")
    else
        print("Invalid username or password")
```

Add basic validation:

```
python
if username == "" OR password == ""
    print("Fields cannot be empty")
```

Static Analysis

Automated technique using specialized tools. Analyzes source code **without executing it**.

Identifies:

- Syntax errors
- Code smells
- Security vulnerabilities
- Standard violations

Can be integrated into **CI/CD pipelines** (Continuous Integration/Continuous Deployment)

Find the Code Smells Activity

Original code:

```
a = 10
b = 20
c = a + b
d = a + b
print(c)
```

Issues:

- **Duplicate calculation** (a + b repeated)
- **Unnecessary variable** (d not used)
- **Poor variable names** (a, b, c)

Improved code:

```
num1 = 10
num2 = 20
sum = num1 + num2
print(sum)
```

Inspection vs Static Analysis

Aspect	Inspection	Static Analysis
Method	Manual	Automated
Focus	Logical correctness	Patterns and rules
Speed	Slower	Faster
Depth	Deeper	Shallower
Scalability	Less scalable	Highly scalable

Both complement each other for better quality.

V&V Goals and Objectives

- Goals are derived from **system requirements**
- Must be **clear, measurable, and achievable**
- Should reflect **user expectations** and **system constraints**

Examples:

- Achieve high test coverage
- Minimize critical defects
- Ensure performance requirements are met

Selecting V&V Techniques

Techniques must be selected based on **project needs**.

Depends on:

- System complexity
- Risk level
- Development model

Applied across **all development phases**.

V&V Techniques by Development Phase

Requirements Phase

- Requirement reviews to check clarity and completeness
- Prototyping to validate user expectations
- Simulation for system behavior understanding
- Traceability to ensure all requirements are covered

Design Phase

- Design reviews (PDR – Preliminary Design Review, CDR – Critical Design Review)
- Architectural validation
- Simulation of system workflows
- Static analysis of design models

Implementation Phase

- Code inspection and walkthroughs
- Static analysis tools for automated checking
- Unit testing for individual components
- Integration testing for combined modules

Maintenance Phase

- Regression testing to ensure stability
- Impact analysis to understand change effects
- Continuous testing in evolving systems

Modern V&V Tools and Approaches

- Automated testing frameworks (e.g., **Selenium**, **JUnit**)
- Continuous Integration and Continuous Deployment (**CI/CD**)
- AI-based testing tools
- Test-Driven Development (**TDD**)
- Behavior-Driven Development (**BDD**)

Roles and Responsibilities in V&V

Proper role definition ensures **accountability** and improves **coordination** among teams.

Main roles:

- Development team
- Independent testing team
- Quality assurance team
- Independent V&V team

Development Team Responsibilities:

- Perform unit and integration testing
- Participate in code reviews and inspections
- Fix identified defects
- Develop prototypes and simulations

Independent Testing Team Responsibilities:

- Conduct system and acceptance testing
- Ensure unbiased evaluation of the system
- Execute formal test plans
- Work parallel with development team

Quality Assurance Team Responsibilities:

- Ensure adherence to standards and processes
- Evaluate software quality continuously
- Verify completeness of documentation and testing
-

Independent V&V Team:

- Provides **objective and external evaluation**
- Used in **safety-critical systems**
- Ensures high level of **reliability and compliance**

Integrating V&V with Development Models

Combines **static and dynamic techniques**. Ensures **continuous quality checks** throughout lifecycle. Aligns with development methodologies.

Waterfall Model:

- Sequential verification at each phase
- Strong documentation and review process

Agile Model:

- Continuous testing in each iteration
- Frequent feedback from users
- Early detection of defects

DevOps:

- Automated testing pipelines
- Continuous monitoring and feedback
- Faster delivery with maintained quality

Problem Tracking

Systematic recording of defects:

- Assigning responsibility for fixing issues
- Monitoring resolution progress

Problem Tracking Tools:

Tool	Purpose
Jira	Issue and project tracking
GitHub Issues	Integrated with code repositories
Bugzilla	Traditional bug tracking system
Redmine	Project management with issue tracking
YouTrack	Supports agile workflows

Progress Monitoring

- Monitoring testing progress **continuously**
- Comparing **planned vs actual** progress
- Identifying **risks and delays** early

Metrics to track:

- Number of test cases executed
- Number of defects identified
- Time and resources used
- Remaining testing effort

Real-World Application Examples

Banking System:

- Requires **high reliability and security**
- Tracks transaction failures, uptime, error rates
- Uses **multiple V&V layers** for assurance

Activity – Identify suitable V&V techniques for:

- A mobile application
- A banking system

Lecture 05

Documenting V&V Strategy, Including Tests and Other Artifacts

Why is Documentation Essential?

Reason	Explanation
Software systems are complex	Involve multiple stakeholders
V&V activities must be consistent	Need structured approach
Quality must be proven	With evidence, not assumptions

Example (Banking System):

- Reports show **what was tested**
- Validation documents confirm **system readiness**

Without documentation → no traceability or accountability

What is a V&V Strategy Document?

A V&V Strategy Document defines:

Question	What It Answers
What	What will be verified and validated
How	Methods (review, testing, etc.)
When	Across lifecycle phases

What artifacts	What documents will be produced
-----------------------	---------------------------------

Example V&V Strategy Table:

Phase	V&V Activity
Requirements	Reviews
Design	Inspections
Coding	Testing
Deployment	Validation

What are Artifacts?

Definition: Documents produced during V&V activities.

Examples of Artifacts:

- Test cases
- Checklists
- Test reports
- Defect reports
- Traceability matrix

Artifacts act as **evidence** that quality checks were performed.

Key V&V Documents (Overview)

Document	Purpose
V&V Plan	Overall strategy
Requirement Traceability Matrix (RTM)	Links requirements to verification
Test Cases	How to test
Test Reports	Results of testing
Defect Reports	Track bugs
Final Validation Report	Confirms system readiness

Each document supports **quality assurance** and **traceability**.

1. Requirement Traceability Matrix (RTM)

Definition: A document that links requirements to verification activities.

Example RTM:

Requirement	Verification Method	Status
Login	Test	Pass
Payment	Review	Pending

Purpose:

- Ensure **all requirements** are verified
- Identify **missing coverage**

2. Test Cases (High-Level)

Definition: A simple scenario describing how to check system functionality.

Example Test Cases:

Test Case	Input	Expected Output
TC01	Valid login	Access granted
TC02	Wrong password	Error message

At this level: Focus is on **what to test**, not detailed steps.

3. Checklists

Definition: A list of items used to verify system quality.

Example (Login Page Checklist):

- Username field exists
- Password is masked

- Error message is displayed
- Input validation is implemented

Purpose:

- Ensure **consistency**
- Reduce **human error**

4. Test Reports

Definition: Documents that summarize results of testing.

Types of Test Reports:

Report Type	Purpose
Test Summary Report	Overall system quality
Test Execution Report	Detailed results
Performance Report	System performance

Example:

20 tests executed → 18 passed, 2 failed

5. Defect Reports

Definition: A document used to record and track bugs.

Example Defect Report:

Bug ID	Description	Severity	Status
B01	Login fails	High	Open

Purpose:

- Track issues
- Support debugging

6. Final Validation Report

Definition: Confirms system meets user requirements.

Includes:

- Validation results
- Risk assessment
- Approval decision

Example:

System tested and validated → ready for deployment

Focus: User satisfaction and system readiness

Traceability (Detailed)

Definition: Links requirement → test → result.

Example:

Requirement	Test Case	Result
Login	TC01	Pass

Purpose:

- Ensure **completeness**
- Support **debugging**

Quick Activity: Match the Document

Question	Answer
a. Ensures all requirements are covered	i. RTM
b. Confirms system readiness	ii. Final Validation Report
c. Shows overall testing results	iii. Test Summary Report

Activity: Login Page Documentation

System: Login Page

Create:

1. **1 test case** (e.g., valid username + password → login success)
2. **2 checklist items** (e.g., password field exists, error message shown)
3. **1 validation point** (e.g., user can successfully log in)

Part 2: Metrics and Measurement

Documentation vs Metrics

Documentation	Metrics
Tells what was built	Tells how well it was built
Description of system	Measurement of system quality

Example:

A system may have:

- **Full documentation**
- **But slow performance**

Metrics help identify such issues **objectively**.

What Are Software Metrics?

Definition: Quantitative measurements used to evaluate software quality.

Common Metrics:

Metric	What It Measures
Defect count	How many bugs

Test coverage	How much code is tested
Response time	How fast the system responds
Reliability rate	How often it works without failure
Usability score	How easy it is to use

Why Important?

They help developers make **data-driven decisions** instead of guessing.

Metric 1: Reliability

Definition: Ability of system to work without failure.

Formula:

text

Reliability = (Total operations - Failures) / Total operations × 100

Example:

- Total operations = 100
- Failures = 5
- Successful operations = 95
- Reliability = $(95 / 100) \times 100 = \mathbf{95\%}$

Interpretation:

Reliability	Meaning
95%	Highly reliable system
70%	Needs improvement

Metric 2: Defect Density

Definition: Measures number of defects per size of software.

Formula:

text

Defect Density = Defects / Lines of Code (LOC)

Example:

- Defects = 20
- LOC = 1000
- Defect Density = $20 / 1000 = 0.02$ defects per LOC

Interpretation:

Value	Meaning
Lower value	Better quality
Higher value	Poor quality

Metric 3: Test Coverage

Definition: Measures how much code is tested.

Formula:

text

Test Coverage = $(\text{Tested items} / \text{Total items}) \times 100$

Example:

- Total test cases = 100
- Tested = 80
- Test Coverage = $(80 / 100) \times 100 = 80\%$

Interpretation:

Coverage	Meaning
100%	Fully tested
<70%	High risk of bugs

Metric 4: Usability

Definition: Measures how easy a system is for users.

Example Scenarios:

Scenario	Usability Level
User logs in in 2 steps; clear navigation menu	Good Usability
Too many confusing buttons; user gets lost	Poor Usability

Measurement Example:

- Task success rate = 90% → **High usability**

Metric 5: Performance

Definition: Measures system speed and efficiency.

Example:

Scenario	Response Time	Quality
Login request	1 second	Good
Report generation	5 seconds	Poor

Formula (basic idea):

Performance = 1 / Response Time

Interpretation:

Lower response time = **better performance**

Activity: Calculate Test Success Rate

Given:

- Total test cases = 50
- Failed test cases = 10

Step 1: Calculate passed tests

- Passed = 50 - 10 = 40

Step 2: Calculate success rate

- Success Rate = $(40 / 50) \times 100$
- Success Rate = **80%**

Discussion Question:

Is 80% acceptable for a banking system? Why or why not?

Suggested answer: Probably **not** acceptable – banking systems require very high reliability (often >99%) because failures can cause financial loss or security breaches.

Lecture 06

V&V Involvement at Different Points in the Life Cycle

Key Principle

V&V is applied throughout the software development lifecycle, **not only at the testing stage**.

Why This Matters:

Reason	Explanation
Errors in early stages	Affect all later stages
Late detection	Increases cost and effort
Continuous verification	Improves overall quality

Example Chain Reaction:

Incorrect requirement → flawed design → incorrect implementation → system failure

V&V Focus by Phase

Phase	V&V Focus	Example
Requirements	Clarity & completeness	Check requirement definition

Design	Structure & architecture	Validate system design
Implementation	Logic correctness	Review code
Integration	Module interaction	Verify communication
Deployment	User needs	Validate real usage

Requirements Phase V&V

What we verify:

- Is the requirement **clear**?
- Is it **measurable**?
- Is it **testable**?

Poor Requirement:

"The system should load quickly"

Improved Requirement:

"The system should load the dashboard within 2 seconds for 90% of users"

Activity: Fixing a Weak Requirement

Original requirement: "The system should handle many users"

Question	Answer
a. Why is this weak?	Vague ("many users" not defined); Not measurable; Not testable
b. Rewritten requirement	"The system shall support at least 1000 concurrent users performing standard operations with no system failure and maintain an average response time of ≤ 2 seconds under normal load conditions."
c. How to verify?	Perform load testing with 1000 concurrent users; Measure response time (≤ 2 seconds); Monitor system stability (no crashes); Use tools like JMeter or LoadRunner

Design Phase V&V

What we verify:

- System architecture
- Module interaction
- Data flow

Example (Online Shopping System):

Product Page → Cart → Payment

Discussion Question:

What happens if payment fails? Is error handling designed?

Implementation Phase V&V

What we verify:

- Code correctness
- Logic
- Coding practices

Example (Poor Code):

```
price = 100
discount = 20
final_price = price - discount
print(final_price)
```

Issues Identified:

- No validation
- Hardcoded values
- Not reusable

Improved Code:

```
price = float(input())
discount = float(input())
if discount <= price:
    print(price - discount)
```

else:

```
print("Invalid discount")
```

Integration Phase V&V

What we verify:

- Do modules work together?
- Is data correctly exchanged?

Example:

Cart → Payment → Order Confirmation

Possible Issues:

- Payment success but no order created
- Data mismatch

Deployment/Validation Phase V&V

What we verify:

- Does system meet user expectations?

Example:

User tries to checkout:

- Is process smooth?

- Is response time acceptable?

Activity: Match the Activity to the Phase

Activity	Phase
1. Detecting logical errors in program code	D. Implementation Phase
2. Validating the system with real users	C. Deployment / Validation Phase
3. Fixing unclear or ambiguous requirements	B. Requirements Phase
4. Checking whether different modules work together correctly	A. Integration Phase
5. Reviewing system architecture and module design	E. Design Phase

Part 2: Reviews and Static Analysis

Types of Reviews (Personal, Peer, Inspection, Walkthrough)

What is a Review?

Definition: A manual evaluation of software artifacts.

Purpose:

- Detect defects early
- Improve quality
- Share knowledge

Personal Review

Definition: Developer reviews their own work.

Example:

- Checking own code
- Reviewing own design

Limitation:

May miss errors (can't see your own blind spots)

Peer Review

Definition: Code reviewed by another developer.

Advantage:

More accurate defect detection (fresh eyes see more)

Discussion: Personal vs Peer Review

Aspect	Personal Review	Peer Review
Speed	Faster	Slower
Accuracy	Lower (bias)	Higher (objective)
Risk	May miss own errors	More thorough

Software Inspection

Definition: Formal, structured review.

Features:

- Roles assigned (moderator, reviewer, recorder)
- Checklist used
- Systematic process

Example:

Team reviews code using a checklist.

Walkthrough

Definition: Author explains the system to the team.

Features:

- Informal
- Interactive
- Feedback-based

Example:

Developer explains the checkout process to the team.

Inspection vs Walkthrough – Comparison

Feature	Inspection	Walkthrough
Formality	High	Low
Focus	Defects	Understanding
Roles	Assigned	Optional
Checklist	Yes	No

Activity: Find the Issue

Code:

```
x = 5
```

```
x = 10
```

```
print(x)
```

Issues Identified:

- Variable x is reassigned before being used

- First assignment (x = 5) is wasted/unused

Improvement: Remove the first assignment.

Part 3: Static Analysis

What is Static Analysis?

Definition: A software verification technique that examines source code **without executing it**.

Key Characteristics:

- No program execution required
- Works on source code, design, or documentation
- Helps improve software quality **before testing begins**

Why is Static Analysis Important?

Benefit	Explanation
Detects defects early	Before execution
Improves code quality	And maintainability
Reduces testing effort	And debugging cost
Supports coding standards	And compliance

What Static Analysis Can Identify:

Issue Type	Example
Unused variables and functions	Variable declared but never used
Duplicate or redundant code	Same calculation repeated
Poor naming conventions	a, b, c instead of meaningful names
Dead or unreachable code	Code that can never be executed
Logical inconsistencies	Contradictory conditions

Static Analysis Example

Code:

```
x = 10
y = 20
z = x + y
z = x + y # Duplicate line
print(z)
```

Issues Identified:

- Duplicate computation (z = x + y repeated)
- Redundant assignment
- Unnecessary code repetition

Activity: Find Unnecessary Code

Code:

```
if x == 0:
    print("zero")
```

Question: What part is unnecessary or can be improved?

Answer: Nothing is strictly unnecessary, but could be improved by handling the else case or adding input validation.

Static Analysis vs Testing – Comparison

Aspect	Static Analysis	Testing
Execution	No execution required	Program execution required
When performed	Early in development	After implementation
What it detects	Code-level issues	Runtime behavior issues
Example	Unused variables, duplicate code	Crashes, wrong outputs

Both complement each other – static analysis finds some issues early, testing finds others later.

Real-World Application

In real software development:

Practice	How It Helps
Code reviews	Detect defects before testing
Static analysis tools	Automate quality checks
V&V process	Ensures reliability across lifecycle
CI/CD pipelines	Widely used in industry for automated checks

CI/CD = Continuous Integration / Continuous Deployment

Lecture 07

What is Static Analysis?

Definition

Static analysis refers to examining source code **without executing it**.

- Performed during **development phase**
- Detects issues **early**
- No program execution required

Simple Definition:

"Static analysis is the process of analyzing source code to find errors without running the program."

Static Analysis vs Dynamic Analysis

Aspect	Static Analysis	Dynamic Analysis
Execution	No execution	Requires execution
When	Early detection	Detects runtime issues
Speed	Faster	Slower
What it finds	Code structure issues	Behavior issues

Why is Static Analysis Important?

Benefit	Explanation
Detects errors early	Before testing begins
Saves time and cost	Fixing early is cheaper
Improves software reliability	Fewer bugs reach production
Enhances security	Finds vulnerabilities early
Maintains coding standards	Enforces consistent style

Part 2: Types of Static Analysis

Two Main Categories:

1. Basic Analysis:

- Lexical Analysis
- Syntax Analysis
- Semantic Analysis

2. Advanced Analysis:

- Control Flow Analysis
- Data Flow Analysis
- Security Analysis
- Coding Standards Analysis

1. Lexical Analysis

What it does: Checks the smallest elements (tokens) of a program – keywords, identifiers, symbols.

Detects:

- Invalid variable names
- Illegal characters

Example:

```
int 1num = 10; // Invalid identifier (cannot start with a number)
```

2. Syntax Analysis

What it does: Checks whether code follows the correct structure and grammar rules of the programming language.

Detects:

- Missing brackets
- Missing semicolons

Example:

```
if(x > 5  
    System.out.println("Hi"); // Missing closing bracket
```

3. Semantic Analysis

What it does: Checks whether the **meaning** of the code is correct.

Detects:

- Type mismatches
- Undeclared variables

Example:

```
int x = "Hello"; // Incorrect data type assignment
```

4. Control Flow Analysis

What it does: Examines the **order of execution** of statements in a program.

Detects:

- Infinite loops
- Unreachable code

Example:

```
while True:
```

```
    pass # Infinite loop
```

5. Data Flow Analysis

What it does: Tracks how **data moves** through variables in a program.

Detects:

- Uninitialized variables
- Unused variables

Example:

```
if x > 5:
```

```
    y = 10
```

```
print(y) # y may be undefined (if x <= 5)
```

6. Security Analysis

What it does: Identifies potential **vulnerabilities** in the code.

Detects:

- SQL injection risks
- Hardcoded sensitive data

Example:

```
query = "SELECT * FROM users WHERE name = '" + user_input + "'"
```

```
# Vulnerable to SQL injection
```

7. Coding Standards Analysis

What it does: Ensures code follows **best practices and conventions**.

Detects:

- Poor naming conventions
- Inconsistent formatting

Example:

```
int A; // Bad naming
```

```
// Better: int age;
```

Summary of Analysis Types

Analysis Type	What It Checks
Lexical	Tokens (keywords, identifiers, symbols)
Syntax	Structure and grammar
Semantic	Meaning and types
Control Flow	Execution path
Data Flow	Variable usage
Security	Vulnerabilities
Standards	Best practices

Part 3: Common Defects Detected by Static Analysis

What Are Common Defects?

Frequent errors found in source code using static analysis.

Examples:

- Syntax errors
- Uninitialized variables
- Dead code
- Security issues

1. Syntax Errors

Errors that occur when code violates programming language rules.

Detects:

- Missing semicolons
- Missing brackets

Example:

```
if(x > 5
    System.out.println("Hello"); // Missing closing )
```

2. Uninitialized Variables

Variables that are used **before assigning a value**.

Detects:

- Variables without initial values
- Risk of unpredictable output

Example:

```
int x;

if(x > 10) {
    printf("Hi");
}

// x is not initialized
```

3. Unused Variables

Variables that are declared but **never used** in the program.

Detects:

- Redundant variables
- Poor code quality

Example:

```
x = 10
y = 20
print(x) # y is never used
```

4. Dead Code (Unreachable Code)

Code that will **never be executed**.

Detects:

- Statements after return
- Impossible conditions

Example:

```
return 5;
System.out.println("Hello"); // Unreachable code
```

5. Division by Zero

An operation where a number is divided by zero, causing errors.

Detects:

- Invalid mathematical operations
- Potential runtime crashes

Example:

```
x = 10 / 0 # Division by zero
```

6. Null Reference Errors

Accessing a variable that has **no value** (null/None).

Detects:

- Missing object initialization
- Unsafe access to variables

Example:

```
x = None
print(x.upper()) # Error: None has no method
```

7. Logical Errors

Errors where the program runs but gives **incorrect results**.

Detects:

- Incorrect conditions
- Wrong calculations

Example:

```
if age > 18:
```

```
    print("Eligible")
```

```
// Should be >= 18 (if age = 18, still eligible)
```

8. Security Vulnerabilities

Weaknesses in code that can be **exploited by attackers**.

Detects:

- SQL injection risks
- Hardcoded passwords

Example:

```
python
```

```
password = "12345" # Hardcoded password
```

9. Type Errors

Errors caused by **incorrect data types**.

Detects:

- Invalid assignments
- Unsupported operations

Example:

```
x = "10" + 5 # Cannot add string and integer
```

Activity: Identify the Defect

For each code snippet, identify:

1. What is the issue?
2. Which defect type?

Example 1:

```
return 10;
```

```
System.out.println("Hi");
```

Answer
Issue: Statement after return will never execute
Defect type: Dead code / Unreachable code
Why: The return statement ends the function, so the print line is never reached

Part 4: Checking Against Formal Specifications

What is a Formal Specification?

Definition: A rule describing correct system behavior.

Example: If input $> 0 \rightarrow$ output must be positive

Checking Against Specifications

Process: Compare code with requirements.

Example:

Specification	Code
output = input $\times$ 2	output = input + 2

Issue: The code does not match the specification (multiplication vs addition).

Activity: Defect Identification

For any code, ask:

1. What is the problem in this code?
2. Which type of defect is this?
3. Which analysis detects it?

Part 5: Static Analysis Tools

Tool	Language/Focus
SonarQube	Multi-language (popular industry tool)
ESLint	JavaScript
Pylint	Python
Checkstyle	Java
PMD	Java, multi-language

Part 6: Advantages and Limitations

Advantages of Static Analysis:

Advantage	Explanation
Early bug detection	Find issues before testing
Improves reliability	Fewer defects in production
Reduces maintenance cost	Cheaper to fix early
Enhances security	Find vulnerabilities early

Limitations of Static Analysis:

Limitation	Explanation
Cannot detect runtime errors	Some bugs only appear when running
May produce false positives	Reports issues that aren't real problems
Requires expertise	Need skill to interpret results
Cannot fully replace testing	Testing is still needed

True/False Questions:

Statement	Answer
Static analysis requires program execution	False (No execution needed)
Static analysis can detect syntax errors	True
Static analysis can detect runtime crashes	False (Only dynamic testing can find runtime crashes)